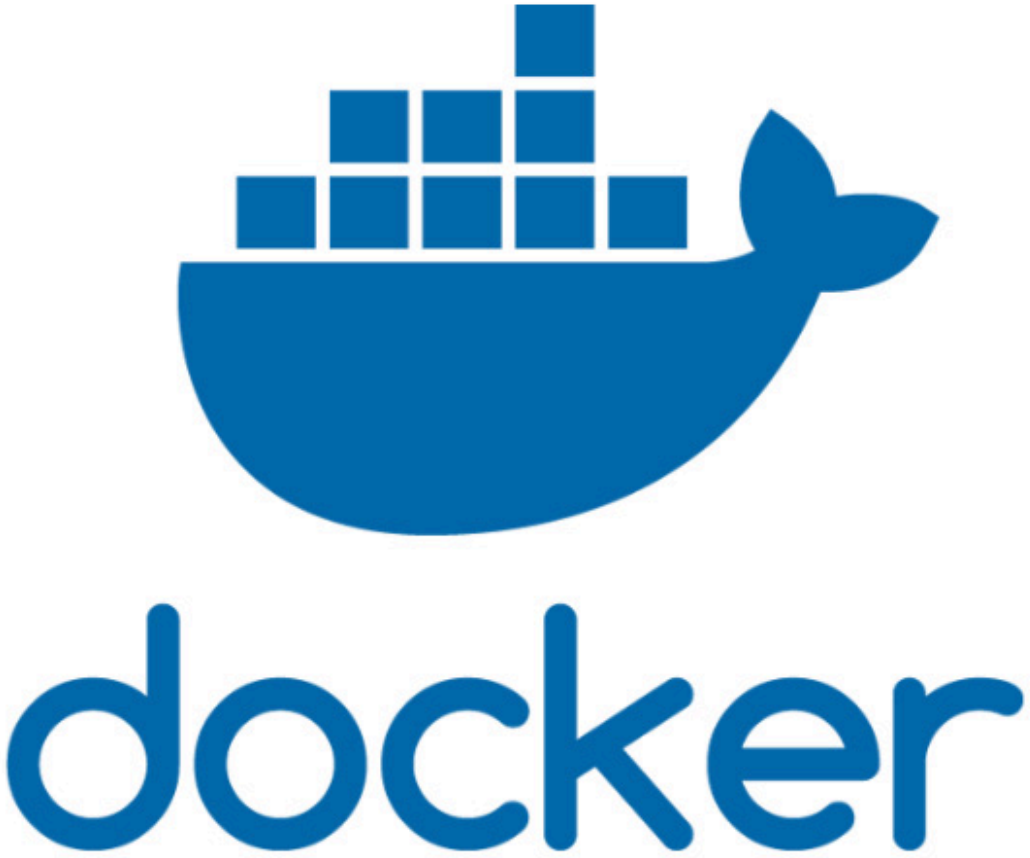
7/21 2회차 스터디



▼ 예습 해오기!

https://github.com/yxzkng/serverstudy-week2/blob/main/docs/01-환경세팅.md
다음주 스터디 자료입니다! 환경세팅 해오시고 카이스트 자료 도커 + 이것이 리눅스가 지정 부분 읽어오시면 감사하겠습니다.

-

https://youtu.be/LXJhA3VWXFA?si=a7y4HKlJBakos9Za
시간 있으신 분은 이 영상도 한번 보고 와주시면 감사하겠습니다

🖥️ Docker로 PetClinic 운영하기

📚 서버 스터디 2주차

https://github.com/likelion-server-study/WEEK2

▼ Chapter 04. 서버를 구축하는 데 알아야 할 필수 개념과 명령

4장은 특정 서버를 직접 구축하는 장이라기보다, 이후 웹·DB·파일·메일 서버 등을 구축하고 운영하기 위해 필요한 **리눅스 관리 도구를 종합적으로 익히는 기초 장**이다.

사용자와 권한 관리, 프로그램 설치, 네트워크 설정, 프로세스와 서비스 관리 등 실제 서버 관리자가 반복해서 사용하는 핵심 작업을 전반적으로 다룬다.

한 줄 요약

본격적으로 서버를 구축하기 전에 필요한 **리눅스 시스템 관리의 기본 개념과 필수 명령어**를 배우는 장이다.

주요 학습 내용

1. 리눅스 시스템 기본 조작

리눅스 시스템을 시작·종료·재부팅하고, 로그아웃하거나 가상 콘솔을 전환하는 방법을 배운다. 런레벨과 `systemd`의 부팅 모드도 살펴본다.

주요 명령어: `shutdown`, `reboot`, `poweroff`, `exit`, `systemctl`

2. 터미널 사용법

명령어 자동 완성, 명령어 기록, 텍스트 에디터, 도움말 사용법을 익힌다.

주요 명령어: `history`, `man`, `vi`

3. 저장장치 연결

CD/DVD, USB와 같은 저장장치를 리눅스 디렉터리에 연결하고 해제하는 마운트 개념을 배운다.

주요 명령어: `mount`, `umount`

4. 사용자와 그룹 관리

여러 사용자가 하나의 서버를 함께 사용할 수 있도록 계정과 그룹을 생성·수정·삭제하고 비밀번호를 관리한다.

주요 명령어: `useradd`, `usermod`, `userdel`, `passwd`, `groupadd`

관련 파일: `/etc/passwd`, `/etc/shadow`, `/etc/group`

5. 파일 소유권과 접근 권한

파일과 디렉터리의 소유자·그룹·허가권을 확인하고 변경하는 방법을 배운다. 하드 링크와 심볼릭 링크의 차이도 다룬다.

주요 명령어: `chmod`, `chown`, `chgrp`, `ln`

6. 프로그램 설치와 관리

리눅스에서 프로그램을 패키지 형태로 설치·삭제·업데이트하는 방법을 배운다.

주요 명령어: `rpm`, `dnf`

7. 파일 관리와 검색

여러 파일을 하나로 묶거나 압축하고, 필요한 파일과 명령어의 위치를 검색하는 방법을 익힌다.

주요 명령어: `tar`, `gzip`, `find`, `which`, `whereis`

8. 작업 자동화

정해진 시간에 명령어가 자동으로 실행되도록 예약하는 방법을 배운다.

주요 명령어: `cron`, `crontab`, `at`

9. 네트워크와 명령어 연결

서버의 네트워크 상태를 확인하고 설정하는 기본 명령어를 익힌다. 파이프, 필터, 리디렉션을 이용해 여러 명령어를 연결하거나 결과를 파일에 저장하는 방법도 배운다.

주요 기능: `|`, `>`, `>>`, `<`

10. 프로세스·서비스·소켓 관리

현재 실행 중인 프로그램인 프로세스를 확인하고 종료하는 방법을 배운다. 백그라운드에서 동작하는 서비스와 필요할 때 서비스를 실행하는 소켓의 개념도 익힌다.

주요 명령어: `ps`, `kill`, `systemctl`

11. 시스템 복구와 부팅 관리

관리자 비밀번호를 잊었거나 시스템이 정상적으로 부팅되지 않을 때 사용하는 응급 복구 방법을 알아본다. GRUB 부트로더와 간단한 커널 컴파일도 다룬다.

▼ 추가 실습 1: Dockerfile로 직접 이미지 빌드하기

이번 실습에서 하는 일

앞에서는 다음 명령으로 PetClinic 이미지를 만들었습니다.

```
./gradlew bootBuildImage --imageName=spring-petclinic:1.0
```

이 명령을 사용하면 Spring Boot와 Buildpacks가 다음 내용을 자동으로 결정합니다.

- 어떤 Java 실행 환경을 넣을지
- PetClinic 파일을 이미지 어디에 배치할지
- 컨테이너를 시작할 때 어떤 명령을 실행할지

서버를 빠르게 컨테이너로 만들기에는 편하지만, 이미지 내부가 어떻게 구성되는지는 직접 확인하기 어렵습니다.

이번에는 Dockerfile을 직접 작성하여 다음 과정을 경험합니다.

```
PetClinic 소스코드
        ↓
실행 가능한 jar 생성
        ↓
Java 실행 환경과 jar를 이미지로 묶기
        ↓
컨테이너로 실행
```

최종적으로 같은 PetClinic 서버를 두 가지 방식으로 이미지화하고, 생성 과정과 결과를 비교합니다.

---

1. PetClinic 프로젝트 폴더로 이동

모든 명령은 자신이 내려받은 `spring-petclinic` 폴더에서 실행합니다.

PetClinic을 홈 폴더에 내려받았다면 다음과 같이 이동합니다.

```
cd ~/spring-petclinic
```

`workspace` 폴더 안에 내려받았다면 다음과 같이 이동합니다.

```
cd ~/workspace/spring-petclinic
```

현재 위치를 확인합니다.

```
pwd
ls
```

다음 파일들이 보이면 실습을 진행할 수 있습니다.

```
build.gradle
gradlew
settings.gradle
src
```

사람마다 PetClinic을 저장한 위치는 달라도 괜찮습니다. 중요한 것은 현재 터미널이 `build.gradle`과 `gradlew`가 있는 PetClinic 최상위 폴더에 있어야 한다는 점입니다.

---

2. Docker 실행 상태 확인

이미지를 만들려면 Docker Desktop의 Docker 엔진이 실행 중이어야 합니다.

```
docker version
```

출력 결과에 `Client`와 `Server` 정보가 모두 나타나는지 확인합니다.

```
Client
 ...

Server
 ...
```

`Client`는 Ubuntu 터미널에서 입력한 Docker 명령을 전달하고, `Server`는 Docker Desktop에서 실제로 이미지와 컨테이너를 관리합니다.

---

3. PetClinic을 실행 가능한 jar로 만들기

Docker 컨테이너에서 PetClinic을 실행하려면 Java 소스코드가 아니라 실행 가능한 프로그램 파일이 필요합니다.

Gradle의 `bootJar` 작업을 사용하여 PetClinic 소스코드와 필요한 라이브러리를 하나의 실행 가능한 jar로 만듭니다.

```
chmod +x gradlew
./gradlew clean bootJar
```

명령	하는 일
`chmod +x gradlew`	Gradle 실행 파일에 실행 권한 부여
`clean`	이전 빌드 결과 삭제
`bootJar`	PetClinic을 실행 가능한 Spring Boot jar로 생성

jar가 생성되었는지 확인합니다.

```
ls -lh build/libs
```

다음과 비슷한 파일이 보이면 성공입니다.

```
spring-petclinic-4.x.x.jar
```

이 jar는 다음 명령만으로 PetClinic을 실행할 수 있는 파일입니다.

```
java -jar build/libs/spring-petclinic-4.x.x.jar
```

이번 실습에서는 이 jar를 직접 실행하지 않고 Docker 이미지 안에 넣어 실행합니다.

---

4. Dockerfile 작성

PetClinic 최상위 폴더에 확장자가 없는 `Dockerfile`을 만듭니다.

```
nano Dockerfile
```

아래 내용을 입력합니다.

```
FROM eclipse-temurin:17-jre

WORKDIR /app
```

```
COPY build/libs/*.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

저장하고 종료합니다.

```
Ctrl + O → Enter → Ctrl + X
```

Dockerfile은 컨테이너에서 PetClinic을 실행하기 위해 필요한 환경과 실행 방법을 순서대로 설명합니다.

Dockerfile 내용	이미지에 적용되는 내용
`FROM eclipse-temurin:17-jre`	Java 17 프로그램을 실행할 수 있는 환경 준비
`WORKDIR /app`	컨테이너 내부 작업 위치를 `/app`으로 지정
`COPY build/libs/*.jar app.jar`	앞에서 만든 PetClinic jar를 이미지에 복사
`EXPOSE 8080`	PetClinic이 컨테이너 내부에서 8080 포트를 사용한다고 표시
`ENTRYPOINT`	컨테이너가 시작되면 `java -jar app.jar` 실행

완성된 이미지에는 다음 두 가지가 함께 들어갑니다.

```
Java 17 실행 환경
+
PetClinic 실행 파일(app.jar)
```

따라서 PetClinic 소스코드나 Gradle이 없는 컴퓨터에서도 Docker만 있으면 동일한 이미지를 실행할 수 있습니다.

---

5. Dockerfile로 이미지 만들기

Docker가 현재 폴더의 Dockerfile을 읽어 이미지를 만들도록 실행합니다.

```
docker build -t spring-petclinic:dockerfile .
```

명령 부분	의미

`docker build`	Dockerfile을 사용해 이미지 생성
`-t`	이미지 이름과 태그 지정
`spring-petclinic`	이미지 이름
`dockerfile`	생성 방식을 구분하기 위한 태그
`.`	현재 폴더를 이미지 생성에 사용할 범위로 지정

마지막의 `.`에는 현재 폴더의 Dockerfile과 `build/libs` 안의 jar를 Docker에 전달한다는 의미가 있습니다.

이미지가 만들어졌는지 확인합니다.

```
docker images | grep spring-petclinic
```

앞에서 Buildpacks 이미지도 만들었다면 두 이미지가 표시됩니다.

```
spring-petclinic   1.0
spring-petclinic   dockerfile
```

이미지	생성 방식
`spring-petclinic:1.0`	Spring Boot Buildpacks가 자동 구성
`spring-petclinic:dockerfile`	직접 작성한 Dockerfile로 구성

---

6. Dockerfile 이미지로 컨테이너 실행

기존 Buildpacks 컨테이너가 내 컴퓨터의 8081 포트를 사용하고 있다면, 이번 컨테이너는 8082 포트를 사용합니다.

```
docker run -d \
  --name petclinic-dockerfile \
  -p 8082:8080 \
  spring-petclinic:dockerfile
```

요청이 전달되는 과정은 다음과 같습니다.

```
브라우저에서 localhost:8082 접속
                ↓
내 컴퓨터의 8082 포트
                ↓
컨테이너 내부의 8080 포트
```

```
                ↓
PetClinic 서버
```

- `p 8082:8080`에서 첫 번째 숫자는 내 컴퓨터의 포트이고, 두 번째 숫자는 컨테이너 내부 PetClinic의 포트입니다.

---

7. 서버 실행 상태 확인

실행 중인 컨테이너를 확인합니다.

```
docker ps
```

다음과 같은 정보가 보이는지 확인합니다.

```
petclinic-dockerfile
0.0.0.0:8082->8080/tcp
```

브라우저에서 PetClinic에 접속합니다.

```
http://localhost:8082
```

PetClinic 화면이 나타나면 Dockerfile로 만든 이미지가 정상적으로 실행된 것입니다.

서버 로그 확인

```
docker logs --tail 100 petclinic-dockerfile
```

실시간으로 로그를 확인합니다.

```
docker logs -f petclinic-dockerfile
```

브라우저에서 PetClinic의 여러 메뉴를 누르면 요청에 따른 로그가 추가되는 것을 확인할 수 있습니다.

로그 확인을 종료할 때는 `Ctrl + C`를 누릅니다. 로그 화면만 종료되며 컨테이너는 계속 실행됩니다.

HTTP 응답 확인

```
curl -i http://localhost:8082/
```

응답 첫 줄에 다음과 같이 `200`이 포함되면 서버가 정상적으로 요청을 처리한 것입니다.

```
HTTP/1.1 200
```

Actuator가 설정된 프로젝트라면 다음 주소도 확인할 수 있습니다.

```
curl -i http://localhost:8082/actuator/health
```

정상적인 예시는 다음과 같습니다.

```
{"status":"UP"}
```

Actuator가 설정되지 않았다면 이 주소에서 `404 Not Found`가 나올 수 있습니다. 이 경우 PetClinic 메인 화면의 HTTP 응답과 컨테이너 로그로 상태를 확인합니다.

---

8. Buildpacks 방식과 Dockerfile 방식 동시에 실행

두 방식으로 만든 이미지가 실제로 동일한 PetClinic 서버를 실행하는지 비교합니다.

Buildpacks 이미지를 8081 포트로 실행합니다.

```
docker run -d \
  --name petclinic-buildpacks \
  -p 8081:8080 \
  spring-petclinic:1.0
```

두 컨테이너를 확인합니다.

```
docker ps
```

접속 주소	실행되는 이미지
`http://localhost:8081`	Buildpacks로 만든 이미지

`http://localhost:8082`	Dockerfile로 만든 이미지

두 주소에서 동일한 PetClinic 화면이 열리는지 확인합니다.

컨테이너 내부에서는 두 PetClinic 모두 8080 포트를 사용하지만, 각 컨테이너의 네트워크 공간이 분리되어 있으므로 충돌하지 않습니다.

내 컴퓨터에서 접속할 때만 8081과 8082로 구분합니다.

---

9. 이미지 크기와 구성 비교

두 이미지의 크기를 확인합니다.

```
docker images \
  --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}" \
  | grep -E "REPOSITORY|spring-petclinic"
```

이미지가 어떤 단계로 만들어졌는지 확인합니다.

```
docker history spring-petclinic:1.0
```

```
docker history spring-petclinic:dockerfile
```

비교 항목	Buildpacks 방식	Dockerfile 방식
이미지 생성	`bootBuildImage` 한 번으로 생성	`bootJar`와 `docker build`로 생성
이미지 구성	Spring Boot와 Paketo가 자동 구성	Dockerfile에 직접 작성
Java 실행 환경	Buildpacks가 자동 선택	`FROM`으로 직접 선택
jar 배치	자동 처리	`COPY`로 직접 지정
서버 실행 명령	자동 처리	`ENTRYPOINT`로 직접 지정
설정 변경	Buildpacks 설정을 알아야 함	Dockerfile을 수정하여 변경
장점	빠르고 일관되게 이미지 생성	이미지 구조를 명확하게 통제

Buildpacks 방식도 내부적으로 Java 실행 환경과 PetClinic 애플리케이션을 이미지 레이어로 나눠 구성합니다.

Dockerfile 방식에서는 이 구성을 직접 작성했기 때문에 이미지 안에 무엇이 들어가고 어떤 명령으로 서버가 시작되는지 더 명확하게 확인할 수 있습니다.

---

10. 실습 결과 확인

확인 항목	성공 기준
jar 생성	`build/libs`에 PetClinic jar가 존재
이미지 생성	`spring-petclinic:dockerfile` 이미지가 표시
컨테이너 실행	`docker ps`에 `petclinic-dockerfile`이 표시
포트 연결	`8082->8080`이 표시
서버 접속	`http://localhost:8082`에서 PetClinic 화면 표시
HTTP 요청	`curl` 결과에 `HTTP/1.1 200` 표시
로그 확인	브라우저 요청에 따라 컨테이너 로그 추가
두 방식 비교	8081과 8082에서 각각 PetClinic 실행
이미지 구성 비교	`docker history` 결과의 레이어 구성이 다르게 표시

11. 실습 종료

실행한 컨테이너를 중지하고 삭제합니다.

```
docker stop petclinic-buildpacks petclinic-dockerfile
```

```
docker rm petclinic-buildpacks petclinic-dockerfile
```

전체 과정 정리

단계	명령	만들어지는 결과
애플리케이션 빌드	`./gradlew clean bootJar`	실행 가능한 PetClinic jar
이미지 빌드	`docker build`	Java와 PetClinic이 포함된 이미지
컨테이너 실행	`docker run`	실제로 요청을 처리하는 PetClinic 서버
상태 확인	`docker ps`, `docker logs`, `curl`	실행·로그·응답 상태
서버 교체	기존 컨테이너 삭제 후 다시 실행	새로운 이미지가 적용된 서버

이번 실습을 완료하면 `bootBuildImage`가 자동으로 처리했던 이미지 구성 과정을 Dockerfile을 통해 직접 확인할 수 있습니다.

참고자료: https://docs.docker.com/reference/dockerfile (도커파일 생성)

▼ 추가 실습 2: 같은 이미지를 서로 다른 사양의 컨테이너로 실행하기

실습에서 확인할 내용

```
PetClinic 이미지 하나 준비
        ↓
서로 다른 자원과 포트 설정으로 컨테이너 두 개 생성
        ↓
각 컨테이너의 접속·로그·자원 사용량 확인
        ↓
한 컨테이너만 중지하여 독립성 확인
        ↓
여러 서버를 하나의 이미지로 운영하는 구조 이해
```

앞에서 만든 `spring-petclinic:1.0` 이미지에는 PetClinic 실행에 필요한 Java와 애플리케이션이 들어 있습니다.

이미지는 한 번 만들면 여러 컨테이너를 실행하는 공통 원본으로 사용할 수 있습니다.

```
spring-petclinic:1.0 이미지
        ├─ 표준 사양 컨테이너
        │   CPU 1 / 메모리 1GB / 호스트 포트 8083
        │
        └─ 경량 사양 컨테이너
            CPU 0.5 / 메모리 512MB / 호스트 포트 8084
```

두 컨테이너는 같은 PetClinic 이미지를 사용하지만 서로 다른 프로세스, 네트워크, 메모리 제한을 가집니다.

이를 통해 실제 운영 환경에서 동일한 서버 이미지를 여러 대 실행하고, 서버마다 자원과 포트를 다르게 설정하는 구조를 확인합니다.

---

1. 실습 이미지 확인

현재 Docker에 PetClinic 이미지가 있는지 확인합니다.

```
docker images | grep spring-petclinic
```

다음 이미지가 보이면 실습을 진행할 수 있습니다.

```
spring-petclinic   1.0
```

이미지가 없다면 PetClinic 프로젝트 폴더에서 먼저 생성합니다.

```
./gradlew bootBuildImage --imageName=spring-petclinic:1.0
```

Docker 명령은 특정 프로젝트 폴더에서만 실행해야 하는 명령이 아닙니다. 이미지가 만들어진 이후의 `docker run`, `docker ps`, `docker stats` 명령은 어느 폴더에서 실행해도 됩니다.

---

2. 표준 사양 컨테이너 실행

먼저 CPU 1개와 메모리 1GB까지 사용할 수 있는 컨테이너를 실행합니다.

```
docker run -d \
  --name petclinic-standard \
  --restart unless-stopped \
  --memory 1g \
  --cpus 1 \
  -p 8083:8080 \
  spring-petclinic:1.0
```

설정	적용되는 내용
`--name petclinic-standard`	컨테이너 이름을 `petclinic-standard`로 지정
`--restart unless-stopped`	사용자가 직접 중지하지 않은 비정상 종료 시 다시 시작
`--memory 1g`	컨테이너가 사용할 수 있는 메모리를 최대 1GB로 제한
`--cpus 1`	컨테이너가 사용할 수 있는 CPU 처리량을 최대 1개 분량으로 제한
`-p 8083:8080`	호스트의 8083 포트를 컨테이너 내부 8080에 연결
`spring-petclinic:1.0`	컨테이너를 만들 때 사용할 이미지

PetClinic은 컨테이너 내부에서 8080 포트를 사용합니다. 브라우저에서는 호스트 포트인 8083으로 접속합니다.

```
localhost:8083
        ↓
petclinic-standard 컨테이너의 8080
        ↓
PetClinic
```

---

3. 경량 사양 컨테이너 실행

같은 이미지로 CPU와 메모리가 더 적은 두 번째 컨테이너를 실행합니다.

```
docker run -d \
  --name petclinic-light \
  --restart unless-stopped \
  --memory 512m \
  --cpus 0.5 \
  -p 8084:8080 \
  spring-petclinic:1.0
```

```
localhost:8084
        ↓
petclinic-light 컨테이너의 8080
        ↓
PetClinic
```

두 컨테이너의 차이는 애플리케이션 코드가 아니라 실행할 때 적용한 설정입니다.

구분	표준 컨테이너	경량 컨테이너
이미지	`spring-petclinic:1.0`	`spring-petclinic:1.0`
CPU 제한	1	0.5
메모리 제한	1GB	512MB
호스트 포트	8083	8084
컨테이너 내부 포트	8080	8080
컨테이너 이름	`petclinic-standard`	`petclinic-light`

- `-memory`와 `-cpus`는 해당 자원을 미리 확보해 두는 설정이 아닙니다. 컨테이너가 지정된 양을 초과해서 사용하지 못하도록 정하는 상한선입니다.

---

4. 두 컨테이너가 실행 중인지 확인

```
docker ps
```

다음과 비슷하게 두 컨테이너가 모두 표시되어야 합니다.

```
petclinic-standard   0.0.0.0:8083->8080/tcp
petclinic-light      0.0.0.0:8084->8080/tcp
```

두 컨테이너가 실제로 같은 이미지에서 만들어졌는지도 확인합니다.

```
docker inspect \
  --format '{{.Name}} → {{.Config.Image}} / image ID={{.Image}}' \
  petclinic-standard petclinic-light
```

두 결과에 다음 이미지 이름과 같은 이미지 ID가 표시되면 같은 원본 이미지로 만든 것입니다.

```
spring-petclinic:1.0
```

이미지는 같지만 컨테이너는 서로 독립적으로 실행됩니다.

---

5. 두 PetClinic 서버에 각각 접속

브라우저에서 다음 두 주소를 엽니다.

```
http://localhost:8083
http://localhost:8084
```

두 주소 모두 PetClinic 화면이 나타나는지 확인합니다.

터미널에서도 HTTP 응답을 확인할 수 있습니다.

```
curl -i http://localhost:8083/
```

```
curl -i http://localhost:8084/
```

두 요청 모두 다음과 같이 `200` 응답을 반환하면 정상입니다.

```
HTTP/1.1 200
```

Actuator가 설정되어 있다면 애플리케이션 상태도 확인합니다.

```
curl -s http://localhost:8083/actuator/health
```

```
curl -s http://localhost:8084/actuator/health
```

정상적인 경우 다음과 같은 결과가 나옵니다.

```
{"status":"UP"}
```

Actuator가 설정되어 있지 않아 `404`가 나온다면 PetClinic 메인 화면의 `200` 응답과 컨테이너 로그를 기준으로 확인합니다.

---

6. 컨테이너별 자원 제한 확인

컨테이너를 실행할 때 설정한 CPU와 메모리 제한을 확인합니다.

```
docker stats --no-stream petclinic-standard petclinic-light
```

출력 결과에서 다음 항목을 비교합니다.

항목	확인할 내용
`CPU %`	현재 컨테이너가 사용하는 CPU 비율
`MEM USAGE`	현재 사용 중인 메모리
`LIMIT`	컨테이너에 설정된 최대 메모리
`MEM %`	메모리 제한 중 현재 사용 비율
`NET I/O`	컨테이너가 주고받은 네트워크 데이터
`BLOCK I/O`	컨테이너의 디스크 입출력

메모리 제한은 다음과 비슷하게 나타납니다.

```
petclinic-standard   ... / 1GiB
petclinic-light      ... / 512MiB
```

동일한 이미지로 실행했어도 각 컨테이너에 서로 다른 자원 제한을 적용할 수 있다는 것을 확인할 수 있습니다.

설정값 자체를 자세히 확인하려면 다음 명령을 사용합니다.

```
docker inspect \
  --format '{{.Name}} memory={{.HostConfig.Memory}} bytes, cpu={{.HostConfig.NanoC
pus}}' \
  petclinic-standard petclinic-light
```

---

7. 요청을 보내면서 자원 사용량 관찰

터미널을 두 개 엽니다.

첫 번째 터미널에서는 자원 사용량을 계속 관찰합니다.

```
docker stats petclinic-standard petclinic-light
```

두 번째 터미널에서는 표준 컨테이너에 동시에 여러 요청을 보냅니다.

```
seq 1 100 \
  | xargs -P 10 -I {} \
    curl -s -o /dev/null -w '%{http_code}\n' \
    http://localhost:8083/ \
  | sort \
  | uniq -c
```

명령의 의미는 다음과 같습니다.

명령 부분	의미
`seq 1 100`	요청 작업 100개 생성
`xargs -P 10`	최대 10개 요청을 동시에 실행
`curl`	PetClinic에 HTTP 요청 전송
`-o /dev/null`	응답 본문은 화면에 출력하지 않음
`-w '%{http_code}\n'`	HTTP 상태 코드만 출력

`sort | uniq -c`	상태 코드별 응답 개수 집계

정상적인 경우 다음과 비슷한 결과가 나옵니다.

```
100 200
```

경량 컨테이너에도 같은 요청을 보냅니다.

```
seq 1 100 \
  | xargs -P 10 -I {} \
    curl -s -o /dev/null -w '%{http_code}\n' \
    http://localhost:8084/ \
  | sort \
  | uniq -c
```

요청을 보내는 동안 첫 번째 터미널의 `docker stats`에서 CPU와 메모리 사용량이 변하는지 관찰합니다.

100개의 간단한 요청만으로 두 컨테이너의 처리 시간이 크게 차이 나지 않을 수도 있습니다. 이번 실습의 핵심은 성능 우열을 정확히 측정하는 것이 아니라 다음 내용을 확인하는 것입니다.

- 같은 이미지로 여러 서버를 실행할 수 있다.
- 컨테이너마다 CPU와 메모리 제한을 다르게 설정할 수 있다.
- 컨테이너별 사용량을 따로 관찰할 수 있다.
- 한 컨테이너의 요청과 자원 사용이 다른 컨테이너와 구분된다.

정확한 성능 측정은 요청 내용, 반복 횟수, 준비 시간 등을 통제하는 별도의 부하 테스트가 필요합니다.

`docker stats` 화면은 `Ctrl + C`로 종료합니다. 컨테이너는 종료되지 않습니다.

---

8. 컨테이너별 로그 확인

각 컨테이너의 로그는 서로 분리되어 저장됩니다.

표준 컨테이너의 로그를 확인합니다.

```
docker logs --tail 50 petclinic-standard
```

경량 컨테이너의 로그를 확인합니다.

```
docker logs --tail 50 petclinic-light
```

실시간으로 표준 컨테이너의 로그를 확인합니다.

```
docker logs -f petclinic-standard
```

이 상태에서 `http://localhost:8083`에 접속하면 표준 컨테이너의 로그에 요청 내용이 추가됩니다.

실시간 로그 확인은 `Ctrl + C`로 종료합니다.

같은 이미지로 만들어진 서버라도 컨테이너별로 상태와 로그가 독립적으로 관리됩니다.

---

9. 컨테이너 하나만 중지하기

경량 컨테이너만 중지합니다.

```
docker stop petclinic-light
```

현재 실행 상태를 확인합니다.

```
docker ps -a \
  --filter name=petclinic-standard \
  --filter name=petclinic-light
```

각 서버에 다시 요청합니다.

```
curl -i http://localhost:8083/
```

```
curl -i --max-time 5 http://localhost:8084/
```

예상되는 결과는 다음과 같습니다.

주소	결과
`localhost:8083`	표준 컨테이너가 계속 실행되어 `200` 응답
`localhost:8084`	경량 컨테이너가 중지되어 접속 실패

두 컨테이너가 같은 이미지에서 만들어졌더라도 실행 상태는 서로 독립적입니다. 경량 컨테이너가 중지되어도 표준 컨테이너는 영향을 받지 않습니다.

경량 컨테이너를 다시 시작합니다.

```
docker start petclinic-light
```

실행 상태와 응답을 다시 확인합니다.

```
docker ps
curl -i http://localhost:8084/
```

- `-restart unless-stopped`를 설정했더라도 사용자가 `docker stop`으로 직접 중지한 컨테이너는 자동으로 재시작되지 않습니다. 위와 같이 `docker start`로 다시 시작해야 합니다.

---

10. 하나의 이미지에서 여러 서버가 실행되는 구조

이번 실습에서 PetClinic 이미지는 하나만 사용했습니다.

```
spring-petclinic:1.0
```

하지만 이 이미지로 두 개의 독립적인 서버 프로세스를 실행했습니다.

```
                    ┌─ petclinic-standard
                    │  CPU 1 / 메모리 1GB
spring-petclinic:1.0│  localhost:8083 → 내부 8080
                    │
                    └─ petclinic-light
                       CPU 0.5 / 메모리 512MB
                       localhost:8084 → 내부 8080
```

두 컨테이너는 다음 항목을 따로 가집니다.

- 컨테이너 이름
- 호스트 포트
- CPU와 메모리 제한
- 실행 상태

- 로그
- 프로세스와 네트워크 공간

반면 애플리케이션 코드와 Java 실행 환경은 동일한 이미지를 사용합니다.

서버 수를 늘릴 때마다 애플리케이션을 새로 빌드하지 않고, 검증된 이미지 하나를 여러 번 실행할 수 있는 구조입니다.

현재는 사용자가 8083과 8084를 직접 구분해서 접속합니다. 이후 Nginx 같은 프록시를 앞에 두면 사용자는 하나의 주소로 접속하고, Nginx가 여러 PetClinic 컨테이너로 요청을 나누도록 구성할 수 있습니다.

```
현재 실습

사용자 ── localhost:8083 ── 표준 컨테이너
사용자 ── localhost:8084 ── 경량 컨테이너


프록시를 추가한 이후

                    ┌─ PetClinic 컨테이너 1
사용자 ── Nginx ────┤
                    └─ PetClinic 컨테이너 2
```

---

11. 실습 결과 확인

확인 항목	성공 기준
같은 이미지 사용	두 컨테이너의 이미지 이름과 이미지 ID가 같음
독립 실행	두 컨테이너가 서로 다른 이름으로 실행됨
포트 분리	8083과 8084에서 각각 PetClinic에 접속 가능
자원 제한	메모리 제한이 각각 1GB와 512MB로 표시
요청 처리	두 주소에서 주로 `200` 응답 반환
상태 분리	한 컨테이너를 중지해도 다른 컨테이너는 계속 응답
로그 분리	컨테이너마다 별도의 로그 확인 가능

---

12. 실습 종료

컨테이너를 중지합니다.

```
docker stop petclinic-standard petclinic-light
```

다음 실습에서도 같은 컨테이너를 사용할 경우에는 삭제하지 않고 다음 명령으로 다시 시작할 수 있습니다.

```
docker start petclinic-standard petclinic-light
```

컨테이너가 더 이상 필요하지 않다면 삭제합니다.

```
docker rm petclinic-standard petclinic-light
```

컨테이너를 삭제해도 `spring-petclinic:1.0` 이미지는 삭제되지 않습니다.

```
docker images | grep spring-petclinic
```

▼ 3주차 미리보기

다음 3주차는 PetClinic 서버를 PostgreSQL과 연결하고, 컨테이너를 다시 실행해도 데이터가 유지되도록 구성해보는 방식으로 진행하려고 합니다.

지금까지 PetClinic은 앱 안에서 빠르게 실행되는 H2 데이터베이스를 사용했습니다. 3주차에는 PostgreSQL을 별도 컨테이너로 실행하고, PetClinic이 해당 DB에 연결되도록 설정합니다.

단순히 DB 연결에 성공하는 것에서 끝나지 않고, 화면에서 추가한 데이터가 실제 DB에 저장되는지, 컨테이너를 다시 실행해도 왜 데이터가 남는지, 여러 DB 변경을 어떻게 한 꺼번에 취소하는지까지 확인해보면 좋겠습니다.

3주차에 확인할 문제

- 앱 안에서 사용하는 H2 DB와 PostgreSQL은 무엇이 다른가

  → H2는 앱 실행을 빠르게 확인하기 위한 개발용 DB에 가깝고, PostgreSQL은 앱 밖에서 별도로 실행되는 실제 DB 서버 역할을 합니다.

- PetClinic이 PostgreSQL을 사용하도록 어떻게 바꾸는가

  → `postgres` profile과 DB 주소·사용자·비밀번호 환경변수를 설정해, 코드 수정 없이 연결할 DB를 바꿉니다.

- 화면에서 추가한 데이터가 실제 DB에 저장되는지 어떻게 확인하는가

  → PetClinic 화면에서 Owner와 Pet을 추가한 뒤, PostgreSQL 내부의 `owners`, `pets`, `visits` 테이블을 직접 조회합니다.

- 컨테이너를 다시 실행해도 데이터가 남게 하려면 어떻게 해야 하는가

  → Named Volume에 PostgreSQL 데이터 파일을 저장하고, 컨테이너를 내렸다가 다시 실행한 뒤에도 추가한 데이터가 남는지 확인합니다.

- 여러 데이터 변경 중 문제가 생기면 어떻게 되돌리는가

  → Transaction의 `BEGIN`과 `ROLLBACK`을 사용해, DB 변경을 임시로 적용했다가 전부 취소하는 과정을 확인합니다.

- DB가 꺼지거나 비밀번호가 틀리면 어디에서 원인을 확인해야 하는가

  → `pg_isready`로 DB 상태를, `/actuator/health`와 PetClinic 로그로 앱 상태를 각각 확인합니다.

▼ 간단한 진행 흐름

1. PostgreSQL 컨테이너 실행

```
docker compose up -d postgres
docker compose ps
docker compose logs --tail 50 postgres
```

1. PetClinic이 PostgreSQL을 사용하도록 환경변수 설정

```
$env:SPRING_PROFILES_ACTIVE="postgres"
$env:POSTGRES_URL="jdbc:postgresql://localhost:5432/petclinic"
$env:POSTGRES_USER="petclinic"
$env:POSTGRES_PASS="petclinic"
```

1. Gradle로 PetClinic 실행

```
.\gradlew.bat bootRun
```

브라우저에서 확인합니다.

```
http://localhost:8080
http://localhost:8080/actuator/health
```

1. PetClinic 화면에서 데이터 추가

구분하기 쉬운 Owner를 한 명 추가합니다.

```
이름: Study
성: VolumeTest
```

가능하면 해당 Owner에게 Pet도 한 마리 추가합니다.

1. PostgreSQL 내부에서 데이터 확인

새 PowerShell 창에서 실행합니다.

```
docker compose exec postgres psql -U petclinic -d petclinic -c "\dt"
docker compose exec postgres psql -U petclinic -d petclinic -c "select * from o
wners where last_name='VolumeTest';"
docker compose exec postgres psql -U petclinic -d petclinic -c "select * from p
ets;"
```

Owner와 Pet의 연결도 확인합니다.

```
docker compose exec postgres psql -U petclinic -d petclinic -c "select o.first_
name, o.last_name, p.name as pet_name from owners o join pets p on o.id = p.own
er_id;"
```

1. DB와 앱 상태를 따로 확인

PostgreSQL 상태:

```
docker compose exec postgres pg_isready -U petclinic -d petclinic
```

PetClinic 상태:

```
curl.exe -i http://localhost:8080/actuator/health
```

- `accepting connections` → DB가 접속 가능한 상태
- `200` 또는 `UP` → PetClinic이 요청에 응답 가능한 상태

1. Named Volume으로 데이터 유지 설정

첨부 매뉴얼의 `compose.override.yml`에 PostgreSQL Volume 설정을 추가합니다.

```
services:
  postgres:
    volumes:
      - petclinic-postgres-data:/var/lib/postgresql/data

volumes:
  petclinic-postgres-data:
```

PetClinic 서버는 `Ctrl+C`로 종료한 뒤 PostgreSQL 컨테이너를 내립니다.

```
docker compose down
docker compose up -d postgres
```

다시 PostgreSQL에 접속해 `VolumeTest` 데이터가 남아 있는지 확인합니다.

```
docker compose exec postgres psql -U petclinic -d petclinic -c "select * from o
wners where last_name='VolumeTest';"
```

컨테이너를 다시 실행한 뒤에도 데이터가 남아 있으면 Volume 설정에 성공한 것입니다.

주의: 아래 명령은 Volume까지 삭제하므로 이번 실습에서는 사용하지 않습니다.

```
docker compose down -v
```

1. Transaction으로 DB 변경 취소하기

`VolumeTest` 데이터를 잠시 수정한 뒤, 실제 저장 전에 변경을 취소해봅니다.

```
docker compose exec postgres psql -U petclinic -d petclinic -c "BEGIN; UPDATE o
wners SET first_name='TempName' WHERE last_name='VolumeTest'; SELECT first_nam
e, last_name FROM owners WHERE last_name='VolumeTest'; ROLLBACK; SELECT first_n
ame, last_name FROM owners WHERE last_name='VolumeTest';"
```

명령어 중간에는 이름이 `TempName`으로 보이지만, `ROLLBACK` 뒤에는 원래 이름인 `Study`로 돌아와야 합니다.

이 실습을 통해 DB는 여러 변경을 묶어서 처리하고, 문제가 생기면 일부만 남기지 않고 한꺼번에 되돌릴 수 있다는 점을 확인합니다.

1. DB 연결 오류 확인

PostgreSQL을 중지합니다.

```
docker compose stop postgres
```

이후 PetClinic의 health 응답과 실행 로그가 어떻게 달라지는지 확인합니다.

```
curl.exe -i http://localhost:8080/actuator/health
```

PostgreSQL을 다시 실행한 뒤 상태를 확인합니다.

```
docker compose start postgres
docker compose exec postgres pg_isready -U petclinic -d petclinic
```

시간이 남으면 `POSTGRES_PASS` 값을 일부러 틀리게 설정하고 PetClinic을 실행해, 비밀번호 오류가 로그에 어떻게 나타나는지도 확인합니다.

3주차 목표는 PostgreSQL 명령어를 외우는 것이 아니라, 앱과 DB를 분리해 연결하고, 화면의 데이터가 DB에 저장되는 구조를 확인하며, Volume과 Transaction을 통해 데이터를 안전하게 유지·처리하는 방식을 직접 이해하는 것입니다.